

Expt. No.: 06	Illustrate multi-tier applications using Node JS and
Date:	Express framework

AIM:

To illustrate a multi-tier application using Node.js and Express by building a Notes app with user login and note management.

PROCEDURE:

- 1 Create a backend server using Node.js and Express.
- 2 Install required packages: Express, Mongoose, CORS, Dotenv, BcryptJS, and JWT.
- 3 Connect the backend to MongoDB using Mongoose.
- 4 Create User and Note schemas in the backend.
- 5 Build authentication routes for Signup and Login.
- 6 Hash user passwords using Bcrypt before storing in database.
- 7 Generate JWT token after successful login/signup.
- 8 Create an authentication middleware to verify JWT token.
- 9 Build protected Note routes:
 - 9.1 Get all notes of logged-in user.
 - 9.2 Add a new note.
 - 9.3 Delete a note.
- 10 Create frontend using React.
- 11 Build Login/Signup form in frontend and store token in localStorage.
- 12 Use Axios to call backend APIs with Authorization header.
- 13 Display notes, add new notes, and delete notes from UI.
- 14 Run backend and frontend servers and test all features.

SOURCE CODE:

In /backend/index.js

```
import express from 'express';
import cors from 'cors';
import mongoose from 'mongoose';
import dotenv from 'dotenv';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
```

```
dotenv.config();

const app = express();
const PORT = process.env.PORT || 5000;
const JWT_SECRET = process.env.JWT_SECRET;

app.use(cors({ origin: 'http://localhost:5173' }));
app.use(express.json());

await mongoose.connect(process.env.MONGO_URI);

const noteSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    content: { type: String, required: true, trim: true },
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  },
  { timestamps: true }
);

const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true, trim: true },
  passwordHash: { type: String, required: true },
});

const User = mongoose.model('User', userSchema);
const Note = mongoose.model('Note', noteSchema);

const createToken = (user) =>
  jwt.sign({ userId: user._id, username: user.username }, JWT_SECRET, { expiresIn:
'7d' });

const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Unauthorized.' });
  }
}
```

```
}

try {
  const payload = jwt.verify(authHeader.slice(7), JWT_SECRET);
  req.user = { id: payload.userId, username: payload.username };
  next();
} catch {
  res.status(401).json({ message: 'Invalid or expired token.' });
}
};

app.post('/api/auth/signup', async (req, res) => {
  const username = req.body.username?.trim().toLowerCase();
  const password = req.body.password?.trim();

  const existingUser = await User.findOne({ username });
  if (existingUser) return res.status(409).json({ message: 'Username is already taken.' });

  const passwordHash = await bcrypt.hash(password, 10);
  const user = await User.create({ username, passwordHash });

  res.status(201).json({ token: createToken(user), user: { id: user._id, username: user.username } });
});

app.post('/api/auth/login', async (req, res) => {
  const username = req.body.username?.trim().toLowerCase();
  const password = req.body.password?.trim();

  const user = await User.findOne({ username });
  const ok = user && (await bcrypt.compare(password, user.passwordHash));
  if (!ok) return res.status(401).json({ message: 'Invalid username or password.' });

  res.json({ token: createToken(user), user: { id: user._id, username: user.username } });
});
```

```
app.get('/api/notes', authMiddleware, async (req, res) => {
  const notes = await Note.find({ userId: req.user.id }).sort({ createdAt: -1 });
  res.json(notes);
});

app.post('/api/notes', authMiddleware, async (req, res) => {
  const { title, content } = req.body;
  const note = await Note.create({ title: title.trim(), content: content.trim(),
userId: req.user.id });
  res.status(201).json(note);
});

app.delete('/api/notes/:id', authMiddleware, async (req, res) => {
  const deleted = await Note.findOneAndDelete({ _id: req.params.id, userId:
req.user.id });
  if (!deleted) return res.status(404).json({ message: 'Note not found.' });
  res.json({ message: 'Note deleted successfully.' });
});

app.listen(PORT, () => console.log('Server running on port ' + PORT));
```

In /frontend/src/App.jsx

```
import { useEffect, useState } from 'react';

const API_BASE_URL = 'http://localhost:5000/api';
const TOKEN_STORAGE_KEY = 'notes_app_token';
const USER_STORAGE_KEY = 'notes_app_user';

function App() {
  const [authMode, setAuthMode] = useState('login');
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [user, setUser] = useState(() =>
JSON.parse(localStorage.getItem(USER_STORAGE_KEY) || 'null'));
  const [token, setToken] = useState(() => localStorage.getItem(TOKEN_STORAGE_KEY) ||
'');

  const [title, setTitle] = useState('');
```

```
const [content, setContent] = useState('');
const [notes, setNotes] = useState([]);

const apiRequest = async (path, method = 'GET', body = null, authToken = '') => {
  const headers = { 'Content-Type': 'application/json' };
  if (authToken) headers.Authorization = 'Bearer ' + authToken;

  const response = await fetch(API_BASE_URL + path, {
    method,
    headers,
    body: body ? JSON.stringify(body) : undefined,
  });

  const data = await response.json().catch(() => ({}));

  if (!response.ok) {
    throw new Error(data.message || 'Request failed');
  }

  return data;
};

useEffect(() => {
  const fetchNotes = async () => {
    if (!token) {
      setNotes([]);
      return;
    }

    try {
      const data = await apiRequest('/notes', 'GET', null, token);
      setNotes(data);
    } catch {
      handleLogout();
    }
  };
});
```

```
    fetchNotes();
  }, [token]);

const saveAuthSession = (nextToken, nextUser) => {
  setToken(nextToken);
  setUser(nextUser);
  localStorage.setItem(TOKEN_STORAGE_KEY, nextToken);
  localStorage.setItem(USER_STORAGE_KEY, JSON.stringify(nextUser));
};

const handleLogout = () => {
  setToken('');
  setUser(null);
  setNotes([]);
  localStorage.removeItem(TOKEN_STORAGE_KEY);
  localStorage.removeItem(USER_STORAGE_KEY);
};

const handleAuthSubmit = async (e) => {
  e.preventDefault();
  const endpoint = authMode === 'signup' ? '/auth/signup' : '/auth/login';
  const data = await apiRequest(endpoint, 'POST', { username, password });
  saveAuthSession(data.token, data.user);
};

const handleAddNote = async (e) => {
  e.preventDefault();
  const data = await apiRequest('/notes', 'POST', { title, content }, token);
  setNotes((prev) => [data, ...prev]);
  setTitle('');
  setContent('');
};

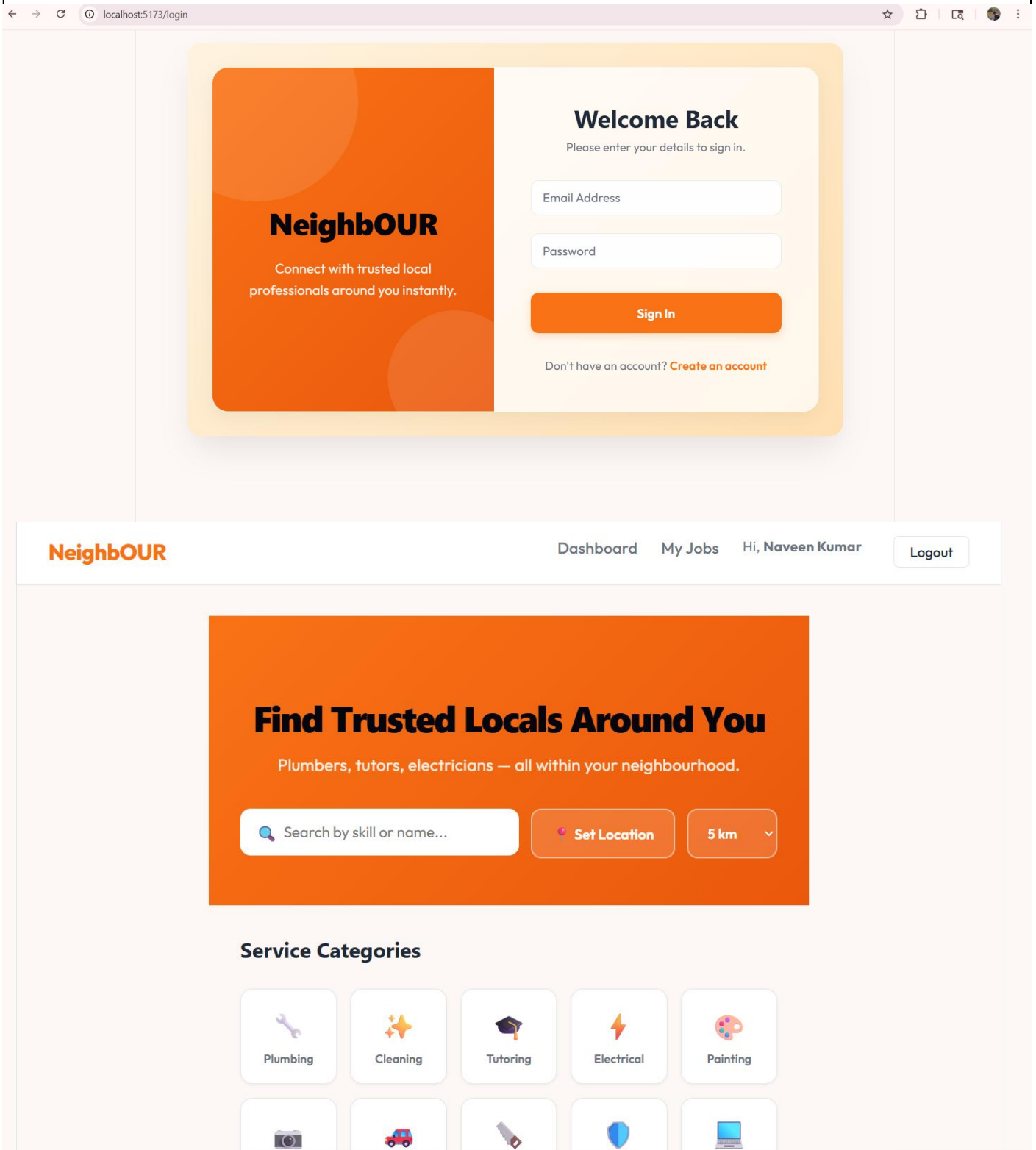
const handleDeleteNote = async (id) => {
  await apiRequest('/notes/' + id, 'DELETE', null, token);
  setNotes((prev) => prev.filter((n) => n._id !== id));
};
```

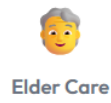
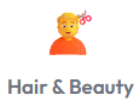
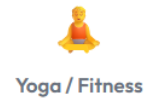
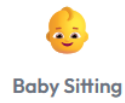
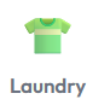
Subject Name: Web Programming
Subject Code: 22CS450

Register Number: 2403917710421054

```
return null;  
}  
  
export default App;
```

Output:





Providers Near You



No providers found

Try increasing your radius or a different location.

Subject Name: Web Programming
Subject Code: 22CS450

Register Number: 2403917710421054

RESULT:

Thus, the program demonstrates a multi-tier application using Node.js, Express, MongoDB, and React.